University of Derby

Bachelor of Science (Honours) in Computing & Information Technology

Module Title: Distributed Applications and Web Services
COURSEWORK ONE COMMENTARY
Module Leader: Dr. Suleiman Aliyu

Name: Anabia Arshad Student ID: 100859795

Originality Statement

I, Anabia Arshad, Student ID: 100859795 declare that this assessment is my own work and that I have correctly acknowledged the work of others using the Harvard referencing. This assessment is in accordance with University guidance on good academic conduct.
Word Count: 1121

Distributed Service Foundation and Remote Invocation Prototype

1. Problem, scope and requirements

The Go-based prototype is used for testing distributed communication, but its two-node scope cannot show all the challenges of decentralised file sharing. It focuses on two independent nodes with unique IDs, configurable addresses and separate local state. Each node supports REST, TCP, UDP and gRPC. JSON and Protocol Buffers are used to represent and exchange data. However, testing these techniques with only two nodes provides limited evidence of how well the system would perform with a large network. Moreover, the current scope does not need Docker, TLS, JWT authentication, gossip-based discovery or secure file-content transfer. Thus, this prototype does not fully address network failures, security risks or the difficulties of managing more nodes. Although it provides a foundation for Coursework Two, these limitations need to be addressed to develop a more reliable and secure file-sharing system.

2. Architecture and platform choice

I have used Go to develop a prototype in which each node operates as an independent process. The command-line entry point manages configuration and startup, whereas separate components handle HTTP requests, network communication, gRPC services and application state. It makes individual behaviours easy to test. In fact, Go provides a suitable platform because its standard library supports HTTP servers, TCP and UDP networking, JSON encoding, concurrency primitives and context-based cancellation. Goroutines also support concurrent request handling without requiring a separate operating-system thread for every operation. It also offers context cancellation, which helps to control request deadlines and shutdown. However, Go does not automatically prevent concurrency errors. When concurrency introduces risks, such as shared maps and other changeable values, data races can take place if access is not properly controlled.

This node architecture separates the command-line entry point, communication handlers, service logic and state management. Each node starts with its own configuration through its communication interfaces and manages its local metadata independently. Clear package boundaries make the system easy to test. It also helps to identify failures within individual components. However, several listeners create additional lifecycle responsibilities. If one listener

fails to start, the application needs to report the error and avoid leaving partially running services. State ownership is another important design choice for this prototype. Both nodes have their own local state. It reduces dependencies. Its main limitation is that developers must manage architectural rules and concurrency carefully.

3. REST and state design

I have implemented REST to expose node health and selected state through HTTP resources. GET retrieves information, whereas POST creates resources or performs defined state-changing operations. The REST interface is more accessible to general HTTP clients than a gRPC interface. These features make REST useful for testing through tools, such as curl. It is also easy to process using Go's standard library. However, REST does not automatically ensure valid data or safe state changes. The application must validate requests, check content types, limit request sizes and return meaningful HTTP status codes. For example, malformed input should produce HTTP 400, whereas an unsupported method should produce HTTP 405. The main architectural consideration is the boundary between transport handling and state ownership. HTTP handlers should validate incoming data and invoke state operations. A concurrency-safe registry can protect shared metadata using a mutex, whereas returning copied snapshots prevents callers from accidentally modifying internal state. Deterministic ordering also makes responses and automated tests more reproducible. The negative test is important because returning the correct error message is not enough if the data has already been damaged. However, REST has certain limitations, for instance, HTTP status codes and JSON structures do not automatically provide authentication, consistency across nodes or protection against concurrent updates.

4. TCP, UDP and gRPC

TCP provides a reliable, ordered byte stream, whereas UDP sends independent datagrams without guaranteeing delivery, ordering or duplicate prevention (Eddy, 2022; Postel, 1980). Their suitability depends on application requirements. For TCP, this prototype uses explicit framing, validates message lengths and rejects oversized input. Deadlines are also considered to prevent connections from waiting indefinitely. TCP is suitable when complete and correctly ordered messages are important, but its connection management adds complexity. A successful

round trip demonstrates that a message can be exchanged and decoded, whereas a malformed or oversized message should be rejected without crashing the node.

On the other hand, UDP is appropriate for this prototype's bounded messaging demonstration, but it would be unsuitable as the sole transport for transferring complete files unless reliability, sequencing and integrity checks are added at the application layer. It preserves message boundaries. Packets can be lost, duplicated or received out of order. The choice between TCP and UDP also affects how failures are handled.

I have also implemented gRPC using a Protocol Buffer service contract. The client requests file metadata from the other node. The returned information demonstrates that the remote invocation works. The .proto definition specifies the available operation and the structure of its request and response messages. A negative test confirms that an unavailable gRPC service returns a clear error within the configured deadline. Context cancellation limits waiting time, but it does not necessarily stop work already running on the server. These limitations show why remote calls require more careful failure handling than local function calls.

5. Testing and failure evidence

I have evaluated this prototype through positive and negative tests across different communication interfaces. A successful REST request confirms that a node can return its health or state. Successful TCP and UDP exchanges demonstrate message transfer between the two processes. The gRPC metadata response confirms that the client can obtain information from the remote node. Negative tests check whether the system rejects malformed or oversized messages and handles unavailable services. These tests are important because a service should not crash or corrupt its state when a request fails. Shutdown testing also checks whether listeners and background operations stop correctly. However, local tests cannot prove production level reliability. Automated assertions, race detection, repeated tests and recorded terminal evidence strengthen the evaluation.

6. Critical evaluation and Coursework Two handoff

This prototype provides a useful foundation by translating communication models into a running two-node application. Its key strengths are clear state ownership, multiple communication demonstrations, testable failure paths and separate interfaces. However, the system is not yet a secure or fully decentralised file-sharing platform. Successful communication does not establish peer authentication, confidentiality or resistance to malicious traffic. These limitations must be addressed before it can operate as a reliable peer-to-peer service.

Coursework Two will introduce event publishing and subscription, followed by containerisation, TLS, authentication and authorisation. Event-based communication will need clear rules for delivery, handling of duplicate events and missed messages. Containers will improve deployment consistency. Security controls will also protect communication and restrict access. Peer discovery and gossip will introduce further challenges, including stale metadata and network failures. The existing interfaces should be retained where appropriate and retested after each change. This approach reduces integration risks and makes faults easy to identify.

References

Eddy, W. (ed.) (2022) Transmission Control Protocol (TCP). RFC 9293. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc9293.html

Google (n.d.) gRPC documentation. Available at: https://grpc.io/docs/

Postel, J. (1980) User Datagram Protocol. RFC 768. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc768.html

The Go Authors (n.d.) Go documentation. Available at: https://go.dev/doc/